Guía de Actividades **Práctico-Experimentales** Nro. 005

1. Datos Generales

Asignatura	Estructura de Datos
Integrantes	- Freddy Ordoñez - Yimmy Angulo - Richard Guaman - Erick Jahirs - Joaquin Moscol
Ciclo	3ro A / B
Unidad	1: Estructuras de datos estáticas y dinámicas
Resultado de aprendizaje de la unidad	R1. Listas Doblemente Enlazadas y Pilas: Navegación Bidireccional y Gestión LIFO de Mínimo Costo Energético
Práctica Nro.	005
Título de la Práctica	Listas Doblemente Enlazadas y Pilas: Navegación Bidireccional y Gestión LIFO de Mínimo Costo Energético
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 22 y viernes 23 de octubre 2026
Horario	Jueves 29 de octubre 10:30 – 11:30 Viernes 30 de octubre 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Silabo	3 horas

2. Objetivo(s) de la Práctica:

- Modear e implementar en **JavaScript (Node.js)** una estructura de Lista Doblemente Enlazada y una Pila **(Stack LIFO)** para la simulación de un módulo de navegación bidireccional y un sistema transaccional de deshacer/rehacer **(*Undo/Redo*)** para el proyecto de gestión satelital sustentable.
- Evaluar experimentalmente la eficiencia temporal de inserción/eliminación en extremos frente a la reestructuración por desplazamiento en arreglos nativos.

- Cuantificar el consumo de memoria Heap generado por el almacenamiento de punteros dobles y mediante métricas del motor V8, garantizando el cumplimiento de criterios de Sostenibilidad Computacional (*Green Computing*).

3. Materiales y reactivos:

- Entorno de ejecución Node.js (versión LTS 20.x o superior) instalado.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación de estructuras dinámicas.
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos.
- Memoria RAM: Mínimo 4 GB.

5. Procedimiento / Metodología

Enfoque PID-2025:

- **ABP (Aprendizaje Basado en Problemas):** Los estudiantes resolverán el reto de diseñar un subsistema de telemetría y auditoría de un satélite sustentable, el cual requiere almacenar eventos en un historial de navegación bidireccional (adelante/atrás) y ejecutar comandos de reversión inmediata de fallos en modo LIFO (*Last In, First Out*).
- **ABI (Aprendizaje Basado en Investigación):** Investigación empírica sobre la huella espacial de los punteros de 64 bits en la V8 Engine y la justificación matemática de la compensación entre espacio y tiempo (*Space-Time Trade-off*).

Tarea 1: Construcción de la Lista Doblemente Enlazada (Navegación Bidireccional)

1. **En la carpeta del proyecto /APE_005_Listas_Pilas, cree el archivo ListaDoble.js.**
2. **Implemente la clase NodoDoble y la clase ListaDoblementeEnlazada garantizando que las referencias cruzadas se actualicen en tiempo constante O1.**

JavaScript

```
/**
 * Representa un Nodo en una Lista Doblemente Enlazada.
 */
class NodoDoble {
  constructor(datos) {
    this.datos = datos; // Carga útil de telemetría satelital
    this.next = null;   // Referencia al siguiente nodo
    this.prev = null;   // Referencia al nodo anterior
  }
}
/**
 * Estructura de Datos Dinámica Doblemente Enlazada.
 */
class ListaDoblementeEnlazada {
  constructor() {
    this.head = null;
    this.tail = null;
    this.longitud = 0;
  }
  /**
   * Inserta un nuevo elemento al final de la lista.
   * Complejidad Temporal: O(1)
   */
  insertarFinal(datos) {
    const nuevoNodo = new NodoDoble(datos);
    if (!this.head) {
      this.head = nuevoNodo;
      this.tail = nuevoNodo;
    } else {
      this.tail.next = nuevoNodo;
      nuevoNodo.prev = this.tail;
      this.tail = nuevoNodo;
    }
    this.longitud++;
```

```
    return nuevoNodo;
  }
  /**
   * Elimina el último elemento de la lista.
   * Complejidad Temporal: O(1)
   */
  eliminarFinal() {
    if (!this.tail) return null;
    const nodoEliminado = this.tail;
    if (this.head === this.tail) {
      this.head = null;
      this.tail = null;
    } else {
      this.tail = this.tail.prev;
      this.tail.next = null;
    }
    nodoEliminado.prev = null;
    this.longitud--;
    return nodoEliminado.datos;
  }
}
module.exports = { NodoDoble, ListaDoblementeEnlazada };
```



Figura 1. `Implementación del Código de la Tarea 1 (ListaDoble.js)`

```
40    /**
41     * Elimina el último elemento de la lista.
42     * Complejidad Temporal: O(1)
43     */
44    eliminarFinal() {
45      if (!this.tail) return null;
46      const nodoEliminado = this.tail;
47      if (this.head === this.tail) {
48        this.head = null;
49        this.tail = null;
50      } else {
51        this.tail = this.tail.prev;
52        this.tail.next = null;
53      }
54      nodoEliminado.prev = null;
55      this.longitud--;
56      return nodoEliminado.datos;
57    }
58  }
59
60  module.exports = { NodoDoble, ListaDoblementeEnlazada };
61
```

Figura 1.1. `Implementación del Código de la Tarea 1 (ListaDoble.js)`

Tarea 2: Implementación de la Pila Transaccional (Stack LIFO) sobre Punteros

1. **Cree el archivo PilaTransaccional.js.**
2. **Desarrolle la Pila de comandos satelitales apoyada en nodos enlazados para evitar la relocalización de memoria de los arreglos dinámicos al hacer push o pop.**

JavaScript

```
const { ListaDoblementeEnlazada } = require('./ListaDoble');

/**
 * Pila (Stack) orientada a Sostenibilidad Computacional.
 * Operaciones fundamentales Push y Pop en O(1) estricto.
 */
class PilaTransaccional {
  constructor() {
    this.lista = new ListaDoblementeEnlazada();
  }

  /**
   * Apila un comando de telemetría en el tope.
   * Complejidad Temporal: O(1)
   */
  push(comando) {
    this.lista.insertarFinal(comando);
  }
```

```
  /**
   * Desapila y retorna el comando en el tope (Undo).
   * Complejidad Temporal: O(1)
   */
  pop() {
    return this.lista.eliminarFinal();
  }

  /**
   * Retorna la cantidad de elementos almacenados.
   */
  obtenerTamano() {
    return this.lista.longitud;
  }
}

module.exports = PilaTransaccional;
```

```
1  const { ListaDoblementeEnlazada } = require('./ListaDoble');
2  /**
3   * Pila (Stack) orientada a Sostenibilidad Computacional.
4   * Operaciones fundamentales Push y Pop en O(1) estricto.
5   */
6  class PilaTransaccional {
7    constructor() {
8      this.lista = new ListaDoblementeEnlazada();
9    }
10   /**
11    * Apila un comando de telemetría en el tope.
12    * Complejidad Temporal: O(1)
13    */
14   push(comando) {
15     this.lista.insertarFinal(comando);
16   }
17   /**
18    * Desapila y retorna el comando en el tope (Undo).
19    * Complejidad Temporal: O(1)
20    */
21   pop() {
22     return this.lista.eliminarFinal();
23   }
24   /**
25    * Retorna la cantidad de elementos almacenados.
26    */
27   obtenerTamano() {
28     return this.lista.longitud;
29   }
30 }
31 module.exports = PilaTransaccional;
```

Figura 2. *`Implementación del Código de la Tarea 2 (***PilaTransaccional.js***)`*

Tarea 3: Experimento Comparativo (Benchmark CPU & Memory Heap)

1. **Cree el archivo BenchmarkNavegacion.js.**
2. **Implemente un script que ejecute $N = 500.000$ inserciones y extracciones al inicio/final comparando un Array nativo de JavaScript frente a la PilaTransaccional.**

JavaScript

```
const PilaTransaccional = require('./PilaTransaccional');

function obtenerMemoriaMB() {
  return (process.memoryUsage().heapUsed / 1024 / 1024).toFixed(2);
}

const N = 500000;
console.log(`=== BENCHMARK DE RENDIMIENTO (N = ${N}) ===\n`);

// 1. EVALUACIÓN CON ARREGLO TRADICIONAL (unshift / shift)
if (global.gc) global.gc();
const memInicialArray = process.memoryUsage().heapUsed;
console.time('Tiempo Array Nativo (unshift/shift)');

const arregloNativo = [];
for (let i = 0; i < N; i++) {
  arregloNativo.unshift({ id: i, log: 'Evento Satelital' }); // O(n)
}
for (let i = 0; i < N; i++) {
  arregloNativo.shift(); // O(n)
}

console.timeEnd('Tiempo Array Nativo (unshift/shift)');
const memFinalArray = process.memoryUsage().heapUsed;
const deltaMemArray = ((memFinalArray - memInicialArray) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Array: ${deltaMemArray} MB\n`);

// 2. EVALUACIÓN CON PILA DE PUNTEROS DINÁMICOS
if (global.gc) global.gc();
const memInicialPila = process.memoryUsage().heapUsed;
console.time('Tiempo Pila Transaccional O(1)');

const pilaSatelital = new PilaTransaccional();
for (let i = 0; i < N; i++) {
```

```
  pilaSatelital.push({ id: i, log: 'Evento Satelital' }); // O(1)
}
for (let i = 0; i < N; i++) {
  pilaSatelital.pop(); // O(1)
}

console.timeEnd('Tiempo Pila Transaccional O(1)');
const memFinalPila = process.memoryUsage().heapUsed;
const deltaMemPila = ((memFinalPila - memInicialPila) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Pila: ${deltaMemPila} MB\n`);
```

```
const PilaTransaccional = require('./PilaTransaccional');
function obtenerMemoriaMB() {
  return (process.memoryUsage().heapUsed / 1024 / 1024).toFixed(2);
}
const N = 500000;
console.log(`=== BENCHMARK DE RENDIMIENTO (N = ${N}) ===\n`);
// 1. EVALUACIÓN CON ARREGLO TRADICIONAL (unshift / shift)
if (global.gc) global.gc();
const memInicialArray = process.memoryUsage().heapUsed;
console.time('Tiempo Array Nativo (unshift/shift)');

const arregloNativo = [];
for (let i = 0; i < N; i++) {
  arregloNativo.unshift({ id: i, log: 'Evento Satelital' }); // O(n)
}
for (let i = 0; i < N; i++) {
  arregloNativo.shift(); // O(n)
}
console.timeEnd('Tiempo Array Nativo (unshift/shift)');
const memFinalArray = process.memoryUsage().heapUsed;
const deltaMemArray = ((memFinalArray - memInicialArray) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Array: ${deltaMemArray} MB\n`);
// 2. EVALUACIÓN CON PILA DE PUNTEROS DINÁMICOS
if (global.gc) global.gc();
const memInicialPila = process.memoryUsage().heapUsed;
console.time('Tiempo Pila Transaccional O(1)');

const pilaSatelital = new PilaTransaccional();
for (let i = 0; i < N; i++) {
  pilaSatelital.push({ id: i, log: 'Evento Satelital' }); // O(1)
}
for (let i = 0; i < N; i++) {
  pilaSatelital.pop(); // O(1)
}
console.timeEnd('Tiempo Pila Transaccional O(1)');
const memFinalPila = process.memoryUsage().heapUsed;
const deltaMemPila = ((memFinalPila - memInicialPila) / 1024 / 1024).toFixed(2);
console.log(`Impacto de Memoria Heap Pila: ${deltaMemPila} MB\n`);
```

Figura 3. `Implementación del Código de la Tarea 3 (BenchmarkNavegacion.js)`

3. **Ejecute el script con la bandera de recolección de basura:**

   Bash

   node --expose-gc BenchmarkNavegacion.js

Método	Tiempo de Arreglo	Tiempo de Pila	Interpretacion
Arreglo nativo (unshift/ shift)	$\approx 5{:}57.907\ (m{:}\,ss.mm)$ $(\approx 352\ s)$	N/A	Es mas lenta en el procesamiento, debido a que tiene que mover muchos elementos
Pila Transaccional	N/A	$\approx 92.759\ ms$ $(\approx 0,093\ s)$	Es mucho mas rapida y consume menos recurso

➔ **Explicación de los resultados obtenidos :**

➔ **Array nativo:** Para realizar las 500.000 operaciones utilizando unshift() y shift(), el programa tardó aproximadamente 5 minutos y 57 segundos. Esto ocurre porque estas operaciones pueden requerir mover los elementos del arreglo, haciendo que el procesamiento sea más lento a medida que aumenta la cantidad de datos.

➔ **Pila Transaccional:** Con push() y pop(), la Pila Transaccional realizó las mismas 500.000 operaciones en aproximadamente 92.759 milisegundos. Al utilizar la Lista Doblemente Enlazada, puede trabajar directamente sobre el extremo de la estructura sin desplazar los demás elementos, manteniendo una complejidad de O(1).

➔ **Nota metodológica sobre los valores N/A**: La notación *"N/A"* (No Aplica) separa de forma independiente el rendimiento de cada estructura. El arreglo nativo procesa datos en bloques contiguos mediante unshift/shift $O(n)$, mientras que la Pila Transaccional opera con nodos enlazados mediante push/pop $O(1)$. Por ello, se evalúan en entornos aislados sin cruzar métricas heterogéneas

➔ **Repositorio de Github Grupal:**

https://github.com/freddyordonez/Portafolio-de-Estructura-de-Datos.git

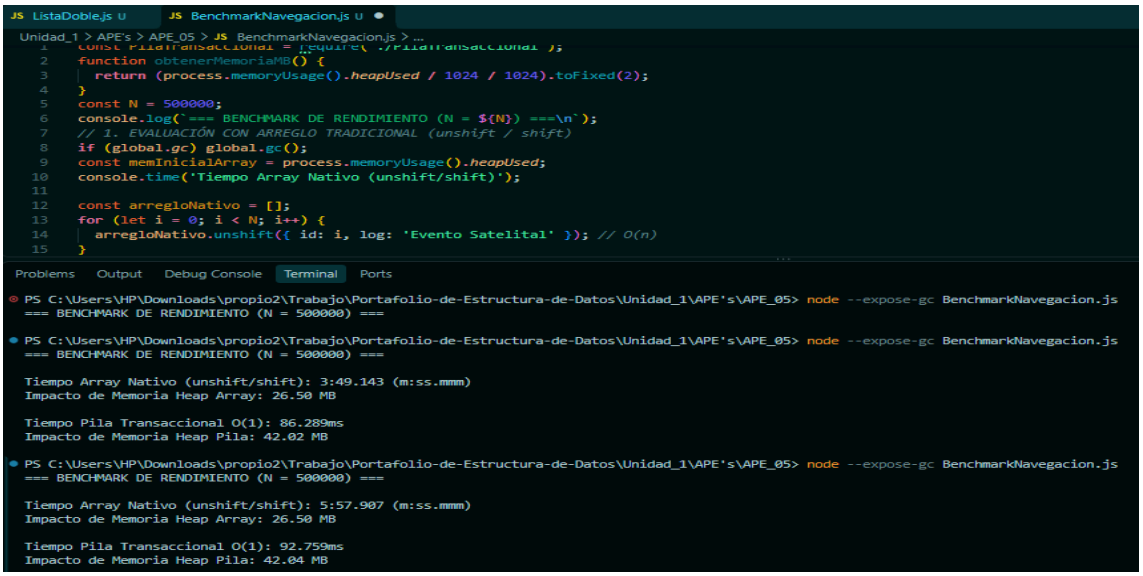
➔ **Nota de ejecución:** La implementación y ejecución de los códigos correspondientes a esta práctica se encuentran debidamente subidas en

las ramas individuales de cada integrante, así como integradas de forma oficial en la rama principal del repositorio grupal.

6. Resultados Esperados

El estudiante evidenciará a través Registro de ejecución en la consola donde se evidencia que las operaciones de inserción y extracción masiva mediante la PilaTransaccional reducen drásticamente el tiempo de ejecución en milisegundos ms en comparación con unshift() y shift() del arreglo nativo, demostrando la diferencia matemática.

➔ **Resultados de la ejecución del archivo BenchMarks.js**



➔ **Gráfica Comparativa de los tiempos de Ejecución entre un Arreglo Nativo y una Pila Transaccional**

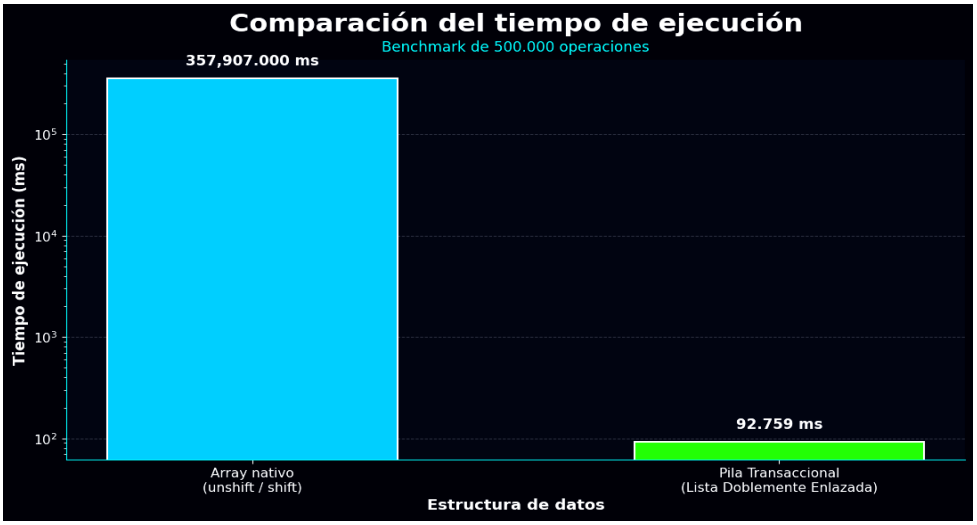


Figura 4. `Gráfico de Barras Comparativo entre Array nativo y Pila Transaccional`

➔ **Explicación de la gráfica de la figura 4:**

Se observa una gran diferencia entre las dos estructuras. El Array nativo necesitó aproximadamente 358 segundos, mientras que la PilaTransaccional realizó las mismas operaciones en aproximadamente 0,093 segundos. Esto demuestra que el uso de una estructura enlazada permite realizar estas operaciones de manera mucho más rápida.

7. Preguntas de Control:

Responda fundamentando sus argumentos bajo una perspectiva lógica, algorítmica y filosófica:

- **En el contexto de las estructuras de datos dinámicas, ¿cuál es la "sustancia" de un nodo en una Lista Doblemente Enlazada en comparación con la de una celda de un arreglo contiguo, y cómo modifica el puntero anterior (prev) su naturaleza constitutiva?**

  La sustancia de una celda en un arreglo contiguo radica en su posición indexada estricta y contigüidad física en memoria, donde su identidad depende del offset respecto al puntero base. En cambio, la sustancia de un nodo en una Lista Doblemente Enlazada es autónoma e independiente de la ubicación física en el Heap, constituida por su carga útil (datos) y sus relaciones contextuales (next y prev). La integración del puntero prev altera fundamentalmente su naturaleza al otorgarle simetría bidireccional: el nodo deja de ser un elemento puramente receptivo en un flujo unidireccional para convertirse en un ente con conciencia contextual completa de su pasado y su futuro inmediatos, permitiendo navegaciones y eliminaciones en $O(1)$ sin requerir el rastreo desde el nodo origen (head).

- **¿Por qué la operación de inserción y extracción LIFO en una Pila implementada mediante punteros dobles alcanza la causa eficiente de la inmediatez O1, superando el trabajo mecánico de reubicación física de celdas que realiza el motor V8 en un arreglo nativo?**

  La eficiencia en tiempo constante de la Pila mediante punteros dobles se debe a que opera sobre nodos enlazados dinámicamente en el Heap, donde las inserciones y extracciones en los extremos solo exigen reasignar referencias puntuales (next y prev) sin alterar la posición física de los elementos existentes. Esto evita por completo el trabajo mecánico del motor V8, el cual se ve obligado a desplazarse masivamente de celda en celda todos los datos contiguos de un arreglo nativo cuando se utilizan métodos como *unshift* o *shift*.

  Mientras el motor V8 sufre un desgaste computacional severo al reubicar 500.000 índices de manera secuencial lo que eleva el

procesamiento a casi 6 minutos, la estructura de punteros dobles aísla cada nodo de forma independiente. Al no requerir relocalización física de memoria contigua en cada operación LIFO, la CPU reduce drásticamente su esfuerzo electromagnético, completando el mismo volumen masivo de transacciones en apenas un fracción de segundo.

- **¿Cuál es el fin último (*telos*) de la navegación bidireccional en un historial de operaciones o sistema de transacciones *Undo/Redo*, y en qué medida optimiza la estabilidad energética del sistema de cómputo?**

  El objetivo de la navegación bidireccional es poder avanzar y retroceder fácilmente en el historial de operaciones, como cuando usamos Undo y Redo. Los punteros next y prev permiten moverse en ambas direcciones sin tener que mover todos los elementos.

  Esto hace que el programa trabaje más rápido y use menos recursos. En la prueba de 500.000 operaciones, se pudo notar una gran diferencia: el arreglo con unshift() y shift() tardó casi 6 minutos, mientras que la PilaTransaccional realizó las mismas operaciones en solo unos 93 milisegundos. Esto demuestra que la pila es mucho más rápida y que la CPU necesita trabajar durante menos tiempo, aprovechando mejor los recursos del computador.

- **Aristóteles sostenía que nada existe sin una forma y materia dada. Si para ganar flexibilidad bidireccional se requiere duplicar las referencias de memoria prev y next, ¿cuándo deja el gasto de memoria Heap de ser una virtud de diseño para convertirse en un exceso ontológico (*overhead*)?**
  Gastar esa memoria extra en el Heap deja de ser algo bueno y se convierte en un desperdicio cuando se hace un programa en el que realmente nunca se va a necesitar ir hacia atrás en los datos, es decir, si solo se va a guardar información que fluye en una sola dirección (como leer una lista siempre hacia adelante), crear ese enlace extra para retroceder (prev) es gastar memoria en vano. Esa "forma" de poder ir en dos direcciones jamás se la va a usar, así que esa "materia" (la memoria RAM de la computadora) simplemente se desperdicia.
  Solo vale la pena gastar ese espacio si de verdad me va a salvar de que el programa se ponga lento al intentar regresar. Si el sistema no necesita ir para atrás, ese enlace extra pierde todo su propósito y se vuelve puro peso muerto para la memoria.

- **Considerando la conservación de energía como principio técnico y ético, ¿de qué manera la eliminación instantánea de referencias sin desplazamiento de elementos mitiga el trabajo electromagnético de la CPU?**

La eliminación de una referencia sin desplazar los demás elementos reduce las operaciones que debe ejecutar la CPU. Por ejemplo, al retirar un nodo de una lista enlazada, una vez localizado, basta con actualizar los enlaces correspondientes; en un arreglo que mantiene el orden, eliminar un elemento puede exigir mover los posteriores.

Esos movimientos implican lecturas y escrituras en memoria, ejecución de instrucciones y cambios de estado en los transistores. Tales cambios requieren cargar y descargar capacitancias eléctricas, lo que consume energía y disipa parte de ella como calor. Evitar desplazamientos innecesarios puede reducir esa actividad eléctrica y, por tanto, el consumo energético. La energía se conserva físicamente; lo que se busca es utilizar menos energía para realizar la tarea.

8. Bibliografía

Liste las fuentes utilizadas. Use normas **IEEE**. Pueden ser libros, manuales, artículos, tutoriales académicos, documentación oficial de software.

[1] B. Goetz, Java Concurrency in Practice. Addison-Wesley, 2022.

[2] P. W. Bible and L. Moser, An Open Guide to Data Structures and Algorithms. PALNI Open Press, 2023.

[3] J. Bloch, Effective Java, 3rd ed. Boston, MA: Addison-Wesley, 2018.

9. Anexos

➔ **Link del repositorio de google colab para visualizar de mejor manera la gráfica de la figura 1 presentada anteriormente:**

https://colab.research.google.com/drive/1YkvtdyYyM1FDU8UV7CNU_J6kBeq2zA1i?usp=sharing

➔ **Repositorio de Github Grupal:**

https://github.com/freddyordonez/Portafolio-de-Estructura-de-Datos.git

➔ **Nota de ejecución:** La implementación y ejecución de los códigos correspondientes a esta práctica se encuentran debidamente subidas en las ramas individuales de cada integrante, así como integradas de forma oficial en la rama principal del repositorio grupal.